

Projektbericht – Memory Controller

1. Aufgabenstellung

Im Rahmen des Grundlagenpraktikums Rechnerarchitektur wurde ein Memory Controller in SystemC implementiert, welcher Zugriffe auf einen Hauptspeicher koordiniert. Es werden drei Funktionen vereint. Die Aufteilung des Adressraums in einen ROM und einen RAM Bereich (Memory Mapping), einen Zugriffsschutz pro Speichereinheit und Nutzer (Memory Protection Unit), sowie ein Lese-/Schreibzugriff der wahlweise mit 1 oder 4 Byte erfolgen kann (Data Width Adaption). Für das Einlesen der CLI Parameter, der ROM Inhalte und der CSV Anfragedateien und das starten der Simulation wurde ein Rahmenprogramm in C angefertigt.

2. Literaturrecherche

Von-Neumann- vs. Harward-Architektur

In einer von-Neuman-Architektur teilen sich Programm und Daten denselben Speicher und Bus. Dadurch müssen Zugriffe immer sequenziell erfolgen. In einer Harward-Architektur werden beide physisch getrennt voneinander behandelt, was zwar parallele Zugriffe erlaubt, allerdings aufwändiger in der Hardware ist [1]. Da der in diesem Projekt entwickelte Memory Controller ROM und RAM über denselben Adressraum und dieselbe Schnittstelle anspricht, entspricht dieser eher einer von-Neumann Architektur.

Einsatzgebiete von Memory Mapping

Memory Mapping wird u.a. bei Memory Mapped I/O eingesetzt. Hier werden Peripheriegeräte (z.B Grafikkarten) über Speicheradressen anstatt spezieller I/O Befehle angesprochen. Da der Zugriff über Pointer erfolgt, wird die Benutzung von Hochsprachen wie C vereinfacht [2].

Fachbegriffe

- **Memory Mapping:** Abbildung logischer Adressen auf physische Speicherbereiche oder Peripherie.
- **Memory Protection Unit:** Verhindert unautorisierte Zugriffe, in diesem Projekt durch Owner-Speicherblock-Zuordnung realisiert.
- **Data Width Adaptation:** Anpassung der Zugriffsbreite von 1 oder 4 Bytes. Der Hauptspeicher unterstützt dabei nur 4 Byte Zugriffe
- **ROM:** Steht für Read-Only-Memory und ist ein rein lesbarer Speicher, der keine Schreibzugriffe erlaubt.

3. Implementierung

Das Projekt gliedert sich in einen C-Teil (Rahmenprogramm) und in einen SystemC-Teil (Hardwaremodule). Die Anfragedatei wird zeilenweise von `csv_parser.c` eingelesen. Jedes Feld (Typ, Adresse, Daten, Nutzer, Zugriffsbreite) wird dabei validiert und es wird ein Array von Requests-Structs erzeugt. Optionale ROM-Inhalte aus einer Textdatei werden von `rom_loader.c` in ein Array von 32-Bit-Worten geladen. In `main.c` werden die CLI-Optionen (`--cycles`, `--tf`, `--latency-rom`, `--rom-size`, `--block-size`, `--rom-content`) verarbeitet und anschließend wird `runSimulation()` aufgerufen.

Das SystemC Modul `MEMORY_CONTROLLER` bildet den Adressraum `[0, romSize)` auf einen internen ROM Vektor ab. Adressen darüber werden nach Zugriffsrechtprüfung an `MAIN_MEMORY` weitergeleitet. Zugriffsrechte werden blockweise in einer `owners-Map` verwaltet. Dabei dürfen Nutzer 0 und 255 immer zugreifen, wobei ein Schreibzugriff von Nutzer 0 einen Block übernimmt und Nutzer 255 ihn wieder freigibt. Für 1 Byte Zugriffe wird zuerst der volle 4 Byte Wert gelesen, das Least-Significant-Byte angepasst und der Wert zurückgeschrieben, um Data-Width-Adaptation korrekt umzusetzen. Im Fall von Fehlern (z.B fehlende Berechtigung, Zugriffe über die ROM/RAM-Grenze hinweg) setzen sofort `error` und `ready` ohne den Hauptspeicher anzusprechen.

Eine Anfrage wie `W, 0x100, 0x11, 1, F` würde also wie folgt verarbeitet werden: Zunächst wird ein Request-Struct von `csv_parser.c` erzeugt. `Simulation.cpp` legt `addr`, `wdata`, `user`, und `wide` auf die Signale und setzt `w`. Im nächsten Takt wird der Schreibwunsch von der `behaviour()`-Methode von `MEMORY_CONTROLLER` erkannt, `ready` auf 0 gesetzt und das Zugriffsrecht des Nutzers auf den jeweiligen Block geprüft. Da es sich um einen 1 Byte Zugriff handelt, wird zunächst der volle 4 Byte Wert aus `MAIN_MEMORY` gelesen, bevor das niederwertigste Byte ersetzt wird und das Ergebnis zurückgeschrieben wird, damit die restlichen drei Bytes erhalten bleiben. Abschließend wird `ready` wieder auf 1 gesetzt.

Zwei Codestellen verwenden hierbei einen wirkungsvollen Trick. In `rangeOverflows` prüft `bytes > 0 && start > UINT32_MAX - (bytes - 1)` einen möglichen Adressüberlauf, ohne dabei selbst einen Überlauf zu erzeugen. In `main.c` wird mit `(x & (x - 1)) == 0` mittels einer einzigen Bitoperation erkannt, ob `--rom-size` eine Zweierpotenz (oder 0) ist.

4. Methodik und Messumgebung

Die Korrektheit wurde mit gezielten CSV Testszenarien, welche alle Spezifikationen abdecken, überprüft: Reine ROM Zugriffe inklusive der erwarteten Fehler bei ROM-Schreibversuchen, Fehler an der ROM/RAM Grenze, byteweise sowie 4 Byte Zugriffe auf über Blockgrenzen hinweg sowie die Zugriffsrechteverwaltung inklusive der Freigabe

eines Blocks durch Nutzer 255. Jeder Testfall wurde über das Rahmenprogramm mit fester Zyklenzahl simuliert. Anschließend wurden die zurückgegebenen result-Werte (cycles, errors) mit den Erwartungswerten verglichen.

5. Persönliche Anteile

Die Aufgabenverteilung erfolgte wie folgt:

- Niklas: Schuf grundlegende Struktur und Überblick über die Implementierung. Legte fest, welche Prüfmechanismen greifen müssen und wann welche Signale gesetzt werden müssen. Außerdem Erstellung des Praktikumsberichts sowie der Präsentation.
- Ahmed: Zuständig für das parsen der Eingabedaten inklusive der Validierung der einzelnen Felder. Korrektur und Anpassung des Praktikumsberichts und der Präsentation.
- Andrei: Zuständig für die SystemC Simulation, also das Anbinden der Signale und die Steuerung des Simulationsablaufs. Korrektur und Anpassung des Praktikumsberichts und der Präsentation.

6. Quellen

- [1] Leitenberger, B.: *Die Harvard und von Neumann Architektur*, <https://www.bernd-leitenberger.de/harvard-neumann.shtml>
- [2] Wikipedia: Memory Mapped I/O, https://de.wikipedia.org/Memory_Mapped_I/O